Stage configs for vLLM-Omni

In vLLM-Omni, the target model is separated into multiple stages, which are processed by different LLMEngines, DiffusionEngines or other types of engines. Depending on different types of stages, such as Autoregressive (AR) stage or Diffusion transformer (DiT) stage, each can choose corresponding schedulers, model workers to load with the Engines in a plug-in fashion.

> **Note**
>
> Default stage config YAMLs (for example, `vllm_omni/model_executor/stage_configs/qwen2_5_omni.yaml` and `vllm_omni/model_executor/stage_configs/qwen3_omni_moe.yaml`) are bundled and loaded automatically when `stage_configs_path` is not provided. They have been verified to work on 1xH100 for Qwen2.5-Omni and 2xH100 for Qwen3-Omni.

Therefore, as a core part of vLLM-Omni, the stage configs for a model have several main functions:

- Claim partition of stages and their corresponding class implementation in `model_executor/models`.
- The disaggregated configuration for each stage and the communication topology among them.
- Engine arguments for each engine within the stage.
- Input and output dependencies for each stage.
- Default input parameters.

If users want to modify some part of it. The custom stage_configs file can be input as input argument in both online and offline. Just like examples below:

For offline (Assume necessary dependencies have ben imported):

```
model_name = "Qwen/Qwen2.5-Omni-7B"
omni = Omni(model=model_name,
stage_configs_path="/path/to/custom_stage_configs.yaml")
```

stable

For online serving:

```
vllm serve Qwen/Qwen2.5-Omni-7B --omni --port 8091 --stage-configs-path
/path/to/stage_configs_file
```

Important

We are actively iterating on the definition of stage configs, and we welcome all feedbacks from both community users and developers to help us shape the development!

Below is a specific example of stage_configs.yaml in Qwen2.5-omni.

```
# stage config for running qwen2.5-omni with AsyncOmniEngine + Orchestrator
runtime.
stage_args:
  - stage_id: 0 # mark the unique id for each stage
    runtime: # The disaggregated configuration
      process: true  # Run this stage in a separate process
      devices: "0" # Visible devices for this stage
(CUDA_VISIBLE_DEVICES/torch.cuda.set_device)
    engine_args: # Engine arguments for a certain engine
      model_stage: thinker
      max_num_seqs: 1
      model_arch: Qwen2_5OmniForConditionalGeneration # The model implementation
registered in model_executor/models/registry.py
      worker_type: ar # The specific worker used
      scheduler_cls: vllm_omni.core.sched.omni_ar_scheduler.OmniARScheduler # The
specific scehduler used
      gpu_memory_utilization: 0.8 # The gpu memory allocation for the stage within
a single chip
      enforce_eager: true  # Now we only support eager mode
      trust_remote_code: true # Needed by huggingface config parsing
      engine_output_type: latent  # It claims that the stage will input latent
hiddenstates besides token ids
      enable_prefix_caching: false # For request with hiddenstates output, the
prefix caching is not supported now
    is_comprehension: true # If the stage is a text or multimodal comprehension
module. If it is, the AsyncOmni will use its tokenizer as default
    final_output: true # If the stage has output as part of final outputs. If it
is false, which means that the stage only works as a intermediate role.
    final_output_type: text # What is the final output type. It can be text and
audio now.
    default_sampling_params: # sampling parameters for the stage. Their meaning
aligns with vLLM.
      temperature: 0.0
      top_p: 1.0
      top_k: -1
      max_tokens: 2048
      seed: 42
      detokenize: True
      repetition_penalty: 1.1
```

stable

```yaml
  - stage_id: 1
    runtime:
      process: true
      devices: "1"
    engine_args:
      model_stage: talker
      max_num_seqs: 3
      model_arch: Qwen2_5OmniForConditionalGeneration
      worker_type: ar
      scheduler_cls: vllm_omni.core.sched.omni_ar_scheduler.OmniARScheduler
      gpu_memory_utilization: 0.8
      enforce_eager: true
      trust_remote_code: true
      enable_prefix_caching: false
      engine_output_type: latent
    engine_input_source: [0]
    custom_process_input_func:
vllm_omni.model_executor.stage_input_processors.qwen2_5_omni.thinker2talker
    default_sampling_params:
      temperature: 0.9
      top_p: 0.8
      top_k: 40
      max_tokens: 2048
      seed: 42
      detokenize: True
      repetition_penalty: 1.05
      stop_token_ids: [8294]
  - stage_id: 2
    runtime:
      process: true
      devices: "0"            # Example: use a different GPU than the previous
stage; use "0" if single GPU
    engine_args:
      model_stage: code2wav
      max_num_seqs: 1
      model_arch: Qwen2_5OmniForConditionalGeneration
      worker_type: generation
      scheduler_cls:
vllm_omni.core.sched.omni_generation_scheduler.OmniGenerationScheduler
      gpu_memory_utilization: 0.15
      enforce_eager: true
      trust_remote_code: true
      enable_prefix_caching: false
      engine_output_type: audio
    engine_input_source: [1]
    final_output: true
    final_output_type: audio
    default_sampling_params:
      temperature: 0.0
      top_p: 1.0
      top_k: -1
      max_tokens: 2048
      seed: 42
```

stable

```
      detokenize: True
      repetition_penalty: 1.1

# Top-level runtime config (concise): default windows and stage edges
runtime:
  enabled: true
  defaults:
    window_size: -1             # Simplified: trigger downstream only after full
upstream completion
    max_inflight: 1             # Simplified: process serially within each stage
  edges:
    - from: 0                   # thinker → talker: trigger only after receiving
full input (-1)
      to: 1
      window_size: -1
    - from: 1                   # talker → code2wav: trigger only after receiving
full input (-1)
      to: 2
      window_size: -1
```

Stage Configuration Arguments

Each stage in the `stage_args` list contains the following configuration options:

`stage_id`

A unique identifier for each stage in the multi-stage pipeline. Stages are numbered sequentially starting from 0, and this ID is used to reference stages in inter-stage dependencies (e.g., `engine_input_source`).

`prompt_expand_func` (Optional)

A custom Python function hook for the LLM stage (Stage 0) that expands a single incoming prompt object into multiple prompts. This is primarily used for multi-modal Classifier-Free Guidance (CFG), where it generates the necessary companion requests (like a negative text prompt) and tags them with internal roles (e.g., `cfg_text`). This ensures the upstream LLM generates the needed contextual hidden states for both the conditional and unconditional generations simultaneously.

`cfg_kv_collect_func` (Optional)

stable

A custom Python function hook for downstream diffusion stages (Stage 1+) to collect, map, and process the KV caches transferred from the companion requests fired by `prompt_expand_func`. It

aggregates the hidden condition states cleanly (e.g., binding them as `cfg_text_past_key_values` and `cfg_text_kv_metadata`), allowing the diffusion runtime to perform CFG smoothly without redundantly evaluating text paths on the DiT workers.

`runtime`

Configuration for disaggregated execution of the stage, controlling how the stage is deployed and executed.

`runtime.process`

Whether to run this stage in a separate process. When set to `true`, the stage will be executed in an isolated process, enabling better resource isolation and parallel execution across different stages. This is essential for multi-GPU deployments where different stages run on different devices.

Default: `true`

`runtime.devices`

Visible devices for this stage, specified as a string. This controls which GPU devices are available to the stage process, similar to setting `CUDA_VISIBLE_DEVICES` or using `torch.cuda.set_device()`. For example, `"0"` uses GPU 0, `"1"` uses GPU 1, and `"0,1"` makes both GPUs 0 and 1 visible.

Default: `"0"`

`engine_args.max_num_seqs`

The maximum number of sequences for concurrent processing in this stage. For LLM stages, this controls the vLLM scheduler's maximum concurrent sequences. For all stage types, this also controls how many tasks can be batched together in the task processing loop.

Default: `1`

`engine_args`

Engine arguments for configuring the LLM engine, diffusion engine, or other engine types used by this stage.

`engine_args.model_stage`

stable

The name identifier for this model stage within the multi-stage architecture. This is used internally to distinguish different stages of the same model (e.g., "thinker", "talker", "code2wav" in Qwen2.5-

Omni).

`engine_args.model_arch`

The model architecture class name that is registered in `model_executor/models/registry.py`. This specifies which model implementation to use for this stage. The class must be registered in the model registry for vLLM-Omni to locate and instantiate it.

`engine_args.worker_cls`

The specific worker class to use for this stage. This determines how the model computations are executed. Examples include `vllm_omni.worker.gpu_ar_worker.GPUARWorker` for autoregressive stages and `vllm_omni.worker.gpu_generation_worker.GPUGenerationWorker` for diffusion-based stages.

`engine_args.scheduler_cls`

The scheduler class to use for this stage. The scheduler manages request queuing, batching, and execution order. Examples include `vllm_omni.core.sched.omni_ar_scheduler.OmniARScheduler` for standard stages and `vllm_omni.core.sched.omni_generation_scheduler.OmniGenerationScheduler` for diffusion stages.

`engine_args.gpu_memory_utilization`

The fraction of GPU memory to allocate for this stage within a single GPU chip. This is a value between 0.0 and 1.0, where 0.8 means 80% of the GPU memory will be used by this stage. This allows fine-grained control over memory allocation when multiple stages share the same GPU or when reserving memory for other operations.

Default: `0.8`

> **Memory Configuration Guide**
>
> For detailed information on how to calculate memory requirements and properly configure `gpu_memory_utilization`, see the GPU Memory Calculation and Configuration Guide.

`engine_args.enforce_eager`

Whether to enforce eager execution mode. When set to `true`, the engine w
without using CUDA graphs or other compilation optimizations. Currently, w
supports eager mode.

stable

Default: `true`

`engine_args.trust_remote_code`

Whether to trust remote code when loading models from Hugging Face. This is required for models that use custom code in their configuration files. Set to `true` when loading models that require custom model implementations.

Default: `true`

`engine_args.engine_output_type`

Specifies the type of output produced by this stage's engine. This determines what kind of data flows to downstream stages. Possible values include `latent` (hidden states), `text` (tokenized text), and `audio` (audio waveforms). When set to `latent`, the stage outputs latent hidden states in addition to token IDs, which are consumed by downstream stages.

Default: `latent`

`engine_args.enable_prefix_caching`

Whether to enable prefix caching for this stage. Prefix caching can improve performance by caching KV cache for common prompt prefixes. However, for requests that output hidden states (when `engine_output_type` is `latent`), prefix caching is not currently supported and should be set to `false`.

Default: `false`

`is_comprehension`

Whether this stage is a text or multimodal comprehension module. When set to `true`, the stage acts as a comprehension module that processes input text or multimodal content. If this is the first comprehension stage, `AsyncOmni` will use its tokenizer as the default tokenizer for the entire pipeline.

Default: `true`

`final_output`

Whether this stage produces output that is part of the final outputs returned to the user. When set to `false`, the stage only works as an intermediate stage, processing data for downstream stages but not contributing directly to the final response.

stable

Default: `true`

`final_output_type`

The type of final output produced by this stage. This specifies what format the output will be in when returned to the user. Currently supported values are `text` (for text generation) and `audio` (for audio generation).

Default: `text`

`default_sampling_params`

Default sampling parameters for this stage. These parameters control the generation behavior and align with vLLM's sampling parameter semantics. These defaults are used when no explicit sampling parameters are provided in the request.

`default_sampling_params.temperature`

Sampling temperature for controlling randomness. Lower values (e.g., 0.0) make the output more deterministic and focused, while higher values increase randomness.

Default: `0.0`

`default_sampling_params.top_p`

Nucleus sampling parameter. Only tokens with cumulative probability mass up to `top_p` are considered. This helps filter out low-probability tokens while maintaining diversity.

Default: `1.0`

`default_sampling_params.top_k`

Top-k sampling parameter. Only the top `k` most likely tokens are considered. Set to `-1` to disable top-k filtering and consider all tokens.

Default: `-1`

`default_sampling_params.max_tokens`

Maximum number of tokens to generate in this stage. This limits the length of the output sequence.

stable

Default: `2048`

`default_sampling_params.seed`

Random seed for reproducible generation. When set, the random number generator will be initialized with this seed to ensure consistent outputs across runs.

Default: `42`

`default_sampling_params.detokenize`

Whether to detokenize the output tokens into text. When set to `true`, token IDs are converted back to readable text strings.

Default: `True`

`default_sampling_params.repetition_penalty`

Penalty applied to tokens that have already appeared in the generated sequence. Values greater than 1.0 discourage repetition, while values less than 1.0 encourage it. A value of 1.0 applies no penalty.

Default: `1.1`

`tts_args` (TTS stages only)

Configuration for Text-to-Speech specific parameters. This section is only applicable to TTS model stages (e.g., `qwen3_tts`).

`tts_args.max_instructions_length`

Maximum character length for voice style/emotion instructions. Instructions exceeding this limit will be rejected with a validation error.

Default: `500`

This value can be overridden at runtime using the `--tts-max-instructions-length` CLI parameter when starting the server.

March 20, 2026

stable